

□ Frontier Exploration Algorithm — Complete Technical Report

Component: `frontier_coordinator.py` — Multi-Robot Frontier Coordinator v2 **Project:** Multi-Robot Autonomous Frontier Exploration **Source:** `frontier_coordinator.py`

Table of Contents

1. What is Frontier-Based Exploration?
 2. System Overview — The Big Picture
 3. Frontier Detection Pipeline
 4. Frontier Deduplication
 5. Frontier Blacklisting
 6. Multi-Robot Assignment — The Cost Matrix
 7. Spatial Partitioning — Region Ownership
 8. Robot State Machine
 9. Stuck Detection & Recovery
 10. Inter-Robot Collision Avoidance
 11. Nav2 Action Client Integration
 12. RViz Visualization Pipeline
 13. All Tuning Parameters
 14. The Replan Loop — Putting It All Together
 15. Comparison: Frontier Coordinator vs. explorelite
 16. Evolution: v1 → v2 Improvements
 17. Deprecated: Single-Robot Frontier Explorer
-

1. What is Frontier-Based Exploration?

The Core Idea

A **frontier** is the boundary between **known free space** and **unknown (unexplored) space** on a map. The fundamental insight — first introduced by Brian Yamauchi in 1997 — is:

If a robot moves toward a frontier, it will observe new territory and expand the known map. By repeatedly navigating to frontiers until none remain, the robot achieves complete exploration of the environment.

In Occupancy Grid Terms

An occupancy grid has three cell states:

Value	Meaning	Visual
0	Free — confirmed empty by LiDAR	White/light gray
100	Occupied — confirmed obstacle	Black/dark
-1	Unknown — never observed	Gray

A **frontier cell** is a **free cell** that is **adjacent to at least one unknown cell**. Clusters of frontier cells form frontier regions, and the **centroid** of each cluster becomes a candidate navigation goal.

-1	-1	-1	-1	-1	-1	-1 = Unknown
-1	-1	-1	-1	-1	-1	
0	0	[F]	[F]	-1	-1	[F] = Frontier (free + adjacent to unknown)
0	0	0	[F]	-1	-1	
0	0	0	0	100	100	0 = Free
0	0	0	0	100	100	100 = Occupied (wall)
□	0	0	0	0	0	□ = Robot

Why Frontiers for Multi-Robot?

For a single robot, frontier exploration is straightforward — go to the nearest/largest frontier. For **two robots**, the challenge is:

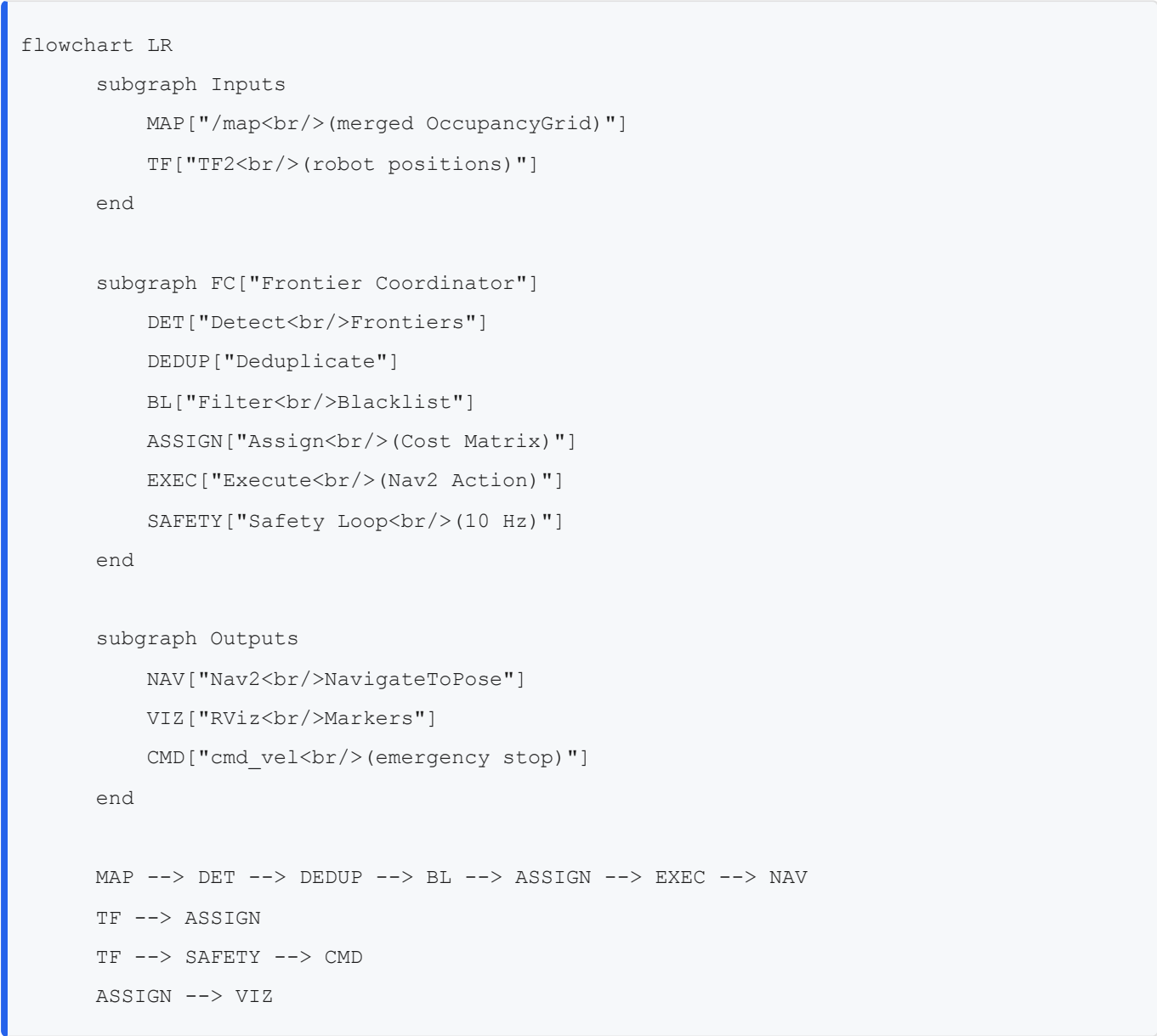
- 1. **Don't send both robots to the same frontier** (wasted effort)
- 2. **Spread robots across the map** (explore faster)
- 3. **Avoid collisions** between robots
- 4. **Handle dynamic maps** (frontiers appear and disappear as exploration progresses)

This is exactly what the Frontier Coordinator solves.



2. System Overview — The Big Picture

Where the Frontier Coordinator Sits



Timing

Loop	Frequency	Purpose
Replan timer	Every 3 seconds	Full pipeline: detect → assign → execute
Safety timer	10 Hz (every 100ms)	Inter-robot collision check
Map subscription	2 Hz (from map_merge)	Stores latest merged map for next replan

*[!IMPORTANT] The frontier coordinator operates on the **merged** map (/map), not on individual robot maps. This means it sees the **combined explored area** of both robots, preventing redundant exploration of already-mapped regions.*

3. Frontier Detection Pipeline

This is the heart of the algorithm — implemented in `_detect_frontiers()`.

Step-by-Step Pipeline

flowchart TB

```
A["1. OccupancyGrid → NumPy<br/>Reshape flat array to (H×W)"] --> B["2. Create Three Binary Masks<br/>unknown_mask, free_mask, obstacle_mask"]
B --> C["3. Morphological Opening<br/>Remove ghost obstacles (3×3 kernel)"]
C --> D["4. Obstacle Inflation<br/>Dilate obstacles by ~0.15m (7×7 ellipse)"]
D --> E["5. Compute Safe Free Mask<br/>free AND NOT inflated_obstacles"]
E --> F["6. Dilate Unknown Mask<br/>Expand unknown by 1 pixel (3×3)"]
F --> G["7. Frontier Mask<br/>safe_free AND dilated_unknown"]
G --> H["8. Connected Components<br/>Cluster frontier pixels"]
H --> I["9. Filter by Area<br/>Keep clusters > 15 pixels"]
I --> J["10. Extract Centroids<br/>Pixel → World coordinates"]
J --> K["11. Compute Map Centroid<br/>(for spatial partitioning)"]
```

Step 1: OccupancyGrid to NumPy

```
grid = np.array(msg.data, dtype=np.int8).reshape((h, w))
```

The incoming `/map` topic contains a flat array of `int8` values. This is reshaped into a 2D grid where:

- `grid[row][col] == -1` → unknown
- `grid[row][col] == 0` → free
- `grid[row][col] >= 50` → occupied (treated as obstacle threshold)

Step 2: Create Binary Masks

Three separate binary masks are created:

```
unknown_mask = np.where(grid == -1, 255, 0).astype(np.uint8)    # White where unknown
free_mask     = np.where(grid == 0, 255, 0).astype(np.uint8)    # White where free
obstacle_mask = np.where(grid >= 50, 255, 0).astype(np.uint8)   # White where obstacle
```

Each mask is a binary image (0 or 255) suitable for OpenCV morphological operations.

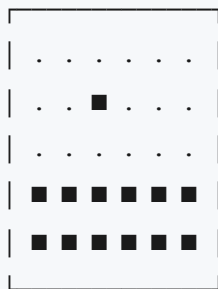
Step 3: Morphological Opening — Ghost Obstacle Removal

```
noise_kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))
obstacle_mask = cv2.morphologyEx(obstacle_mask, cv2.MORPH_OPEN, noise_kernel)
```

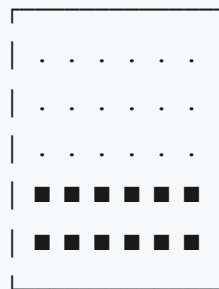
Problem: When one robot's LiDAR scans the other robot, it registers as an obstacle in the SLAM map, creating "ghost obstacles" — small clusters of occupied cells that aren't real walls.

Solution: Morphological opening (erode → dilate) with a 3×3 rectangular kernel removes small isolated obstacle clusters while preserving larger real obstacles (walls).

Before Opening:



After Opening:



← ghost removed!

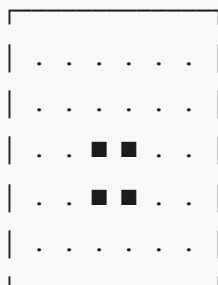
← real wall preserved

Step 4: Obstacle Inflation — Safety Margin

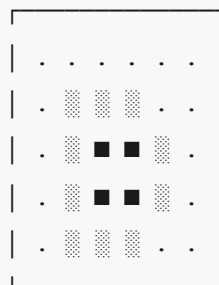
```
obs_kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (7, 7))
dilated_obstacles = cv2.dilate(obstacle_mask, obs_kernel, iterations=1)
```

Purpose: Expands obstacles by approximately **0.15 meters** (at 0.05m/pixel resolution, $7 \div 2 \approx 3$ pixels $\times$ 0.05m $\approx$ 0.15m). This ensures detected frontiers are at a safe distance from walls — preventing Nav2 from receiving goals that are inside or too close to obstacles.

Before Inflation:



After Inflation:



▒ = inflated zone

(no frontiers here!)

Step 5: Safe Free Mask

```
safe_free_mask = cv2.bitwise_and(free_mask, cv2.bitwise_not(dilated_obstacles))
```

This removes any free cells that are within the inflated obstacle zone. Only cells that are both free AND safely away from obstacles can be frontier candidates.

Step 6: Dilate the Unknown Mask

```
kernel = np.ones((3, 3), np.uint8)
dilated_unknown = cv2.dilate(unknown_mask, kernel, iterations=1)
```

Purpose: Expanding the unknown region by 1 pixel ensures that free cells **adjacent** to unknown cells are captured. Without dilation, only cells exactly touching unknown cells would qualify — this accounts for edge effects.

Step 7: Compute the Frontier Mask

```
frontier_mask = cv2.bitwise_and(safe_free_mask, dilated_unknown)
```

This is the key operation: A pixel is a frontier if and only if it is:

1. ☐ Free space (`free_mask`)
2. ☐ Not too close to an obstacle (`safe_free_mask`)
3. ☐ Adjacent to unknown space (`dilated_unknown`)

safe_free_mask:		dilated_unknown:		frontier_mask:
<pre> ■ ■ ■ . . ■ ■ ■ ■ . ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ </pre>	AND	<pre> ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ . ■ ■ ■ ■ . . . ■ ■ </pre>	=	<pre> ■ ■ ■ . . ← frontiers! . ■ ■ ■ ■ ■ </pre>

Step 8: Connected Components Analysis

```
num_labels, _labels, stats, centroids = cv2.connectedComponentsWithStats(frontier_mask)
```

OpenCV's connected components algorithm groups adjacent frontier pixels into discrete clusters. For each cluster it computes:

- `stats[i, cv2.CC_STAT_AREA]` — the number of pixels in the cluster
- `centroids[i]` — the (x, y) pixel coordinate of the centroid

Step 9: Area Filtering

```
for i in range(1, num_labels): # skip label 0 (background)
    if stats[i, cv2.CC_STAT_AREA] > self.min_frontier_size: # > 15 pixels
        # Keep this frontier
```

Small frontier clusters (< 15 pixels $\approx 0.0375 \text{ m}^2$ at 0.05m resolution) are discarded as noise. This prevents the coordinator from sending robots to investigate tiny gaps or sensor artifacts.

Step 10: Pixel $\rightarrow$ World Coordinate Conversion

```
cx, cy = centroids[i]
world_x = origin_x + cx * resolution
world_y = origin_y + cy * resolution
```

Each surviving cluster's centroid is converted from pixel coordinates to world-frame (meters) coordinates using the map's origin and resolution.

Step 11: Map Centroid Computation

```
free_ys, free_xs = np.where(grid == 0)
if len(free_xs) > 0:
    mcx = origin_x + np.mean(free_xs) * resolution
    mcy = origin_y + np.mean(free_ys) * resolution
    self.map_centroid = np.array([mcx, mcy])
```

The centroid of all known free space is computed for use in spatial partitioning (Section 7).

4. Frontier Deduplication

Implemented in `_deduplicate()`.

Problem

Connected components can produce multiple centroids that are very close together, especially along long frontier edges. Sending a robot to one would effectively cover all of them.

Solution

Greedy deduplication with a **1.5-meter radius**:

```
@staticmethod
def _deduplicate(frontiers, radius):
    if not frontiers:
        return []
    unique = [frontiers[0]]
    for f in frontiers[1:]:
        if all(np.linalg.norm(f - u) > radius for u in unique):
            unique.append(f)
    return unique
```

For each frontier centroid, if it's within 1.5m of any already-accepted centroid, it's discarded. This ensures the coordinator doesn't waste assignments on nearly-identical goals.

```
Before Dedup (5 centroids):      After Dedup (2 centroids):
  . .                               .
  .                               .
  .                               .
  . .                               .
  . .                               .
```

5. Frontier Blacklisting

Implemented in the replan loop.

Problem

Sometimes Nav2 rejects a goal or fails to navigate to it (e.g., the goal is unreachable due to obstacles). Without blacklisting, the coordinator would keep re-assigning the same failing frontier every 3 seconds, creating an infinite loop.

Solution

When Nav2 rejects or fails a goal, the frontier coordinates are added to a blacklist with a **30-second TTL**:

```
self.blacklisted_frontiers[tuple(ctx.goal)] = self.get_clock().now().nanoseconds / 1e9
```

During each replan, blacklisted frontiers are filtered out:

```

for f in frontiers:
    for bf, timestamp in list(self.blacklisted_frontiers.items()):
        if current_time - timestamp > self.blacklist_duration: # 30s expired
            del self.blacklisted_frontiers[bf] # remove stale entry
            continue
        if np.linalg.norm(f - np.array(bf)) < 1.0: # within 1m of blackli
sted
            is_blacklisted = True
            break

```

After 30 seconds, the blacklist entry expires. This is important because the map changes over time — a previously unreachable frontier may become reachable as more of the map is explored.

6. Multi-Robot Assignment — The Cost Matrix

Implemented in the replan loop.

The Assignment Problem

Given N robots needing goals and M available frontiers, how do we assign each robot to a frontier to minimize total cost while maximizing exploration efficiency?

Cost Function

For each (robot, frontier) pair, the cost is:

```
total_cost = distance_cost + separation_cost + region_cost
```

Component 1: Distance Cost

```
dist_cost = np.linalg.norm(robot_position - frontier_position)
```

Simple Euclidean distance from the robot to the frontier. Closer frontiers are cheaper.

Component 2: Separation Cost

```
sep_cost = 0.0
for other_robot in all_robots:
    if other_robot != current_robot and other_robot.goal is not None:
        d_other = np.linalg.norm(frontier - other_robot.goal)
        sep_cost += separation_weight / max(d_other, 0.1) # weight = 50.0
```

Purpose: Penalizes sending a robot to a frontier that is close to where the **OTHER** robot is already headed. This pushes robots apart.

Math: The penalty is the **inverse** of the distance to the other robot's goal, scaled by a weight of 50.0. If a frontier is 1m from the other robot's goal: $50 / 1.0 = 50.0$ penalty. If it's 5m away: $50 / 5.0 = 10.0$ penalty.

Component 3: Region Cost

```
reg_cost = self._region_cost(robot_name, frontier)
```

See Section 7 for the full spatial partitioning explanation. In brief: penalizes frontiers on the **other robot's side** of the map.

Minimum Distance Filter

```
if dist_cost < self.min_goal_distance: # 0.6m
    costs.append(float('inf')) # Skip - too close to assign
```

Frontiers within 0.6m of the robot are skipped — the robot is already there and exploring that area.

Assignment Strategy: Greedy Argmin

```
idx = int(np.argmin(costs))
target = available.pop(idx)
```

Robots needing goals are processed sequentially. Each picks the frontier with the **minimum total cost**, and that frontier is removed from the available pool.

*[!NOTE] This is a **greedy** assignment, not a globally optimal one (which would require the Hungarian algorithm). Greedy works well for 2 robots because once robot1 picks a frontier, robot2 inherently gets a different one. The separation cost ensures they don't pick nearby frontiers.*

Fallback: Farthest Frontier

```
if not costs or all(c == float('inf') for c in costs):
    dists = [np.linalg.norm(pos - f) for f in available]
    idx = int(np.argmax(dists)) # Pick the FARTHEST frontier
```

If all costs are infinite (all frontiers are too close or blacklisted), the robot is assigned the **farthest** frontier as a heuristic — move to the least-explored area.

7. Spatial Partitioning — Region Ownership

Implemented in `_region_cost()`.

The Problem

Without spatial partitioning, both robots tend to cluster in the same area because nearby frontiers have lower distance costs. Even with separation cost, they may end up exploring the same region from slightly different angles.

The Solution: Perpendicular Bisector Partitioning

The map is divided into two halves using a **perpendicular bisector** between the robots' **home (initial) positions**:



How It Works

1. **Lock home positions:** The first time both robots are >1m apart (i.e., `map_merge` has aligned them correctly), their positions are recorded as "home positions" and never updated again.
2. **Compute the bisector:**

- Midpoint = $(\text{home1} + \text{home2}) / 2$
- Axis = unit vector from home1 to home2

3. Project each frontier:

```
frontier_offset = frontier_pt - midpoint
projection = np.dot(frontier_offset, to_other_unit)
```

- Positive projection = frontier is on the **other robot's side** → penalize
 - Negative/zero projection = frontier is on **this robot's side** → no penalty

4. **Penalty:** $\max(0.0, \text{projection}) \times \text{region_weight}$ where $\text{region_weight} = 50.0$

Visual Example

```
Robot1 home at (-2, 0), Robot2 home at (2, 0)
Midpoint = (0, 0), Axis = (1, 0) (pointing left→right)

Frontier at (-3, 1): projection onto axis = -3 (robot1's side)
→ Robot1 penalty:  $\max(0, -3) \times 50 = 0$  □ (no penalty)
→ Robot2 penalty:  $\max(0, +3) \times 50 = 150$  □ (heavy penalty)

Frontier at (3, -1): projection onto axis = 3 (robot2's side)
→ Robot1 penalty:  $\max(0, +3) \times 50 = 150$  □ (heavy penalty)
→ Robot2 penalty:  $\max(0, -3) \times 50 = 0$  □ (no penalty)
```

Why Home Positions (Not Current Positions)?

Using **current** positions for partitioning causes a critical instability: as robots move toward each other, the bisector flips rapidly, causing constant goal reassignment. Home positions provide **stable, predictable** partitioning for the entire exploration session.

8. Robot State Machine

Each robot is tracked by a `RobotContext` object with the following state machine:

```
stateDiagram-v2
    [*] --> IDLE: Node starts

    IDLE --> NAVIGATING: Frontier assigned<br/>Nav2 goal sent via action client

    NAVIGATING --> IDLE: Goal reached<br/>(robot within 0.4m of goal)
    NAVIGATING --> IDLE: Frontier vanished<br/>(not within 2.5m of any current fronti
er<br/>AND min_commitment_time elapsed)
    NAVIGATING --> IDLE: Stuck detected<br/>(< 0.1m movement for 3 replan cycles)
    NAVIGATING --> IDLE: Nav2 goal failed<br/>(frontier blacklisted for 30s)
    NAVIGATING --> IDLE: Nav2 goal rejected<br/>(frontier blacklisted for 30s)
    NAVIGATING --> PAUSED: Collision risk<br/>(< 0.6m from other robot<br/>AND furthe
r from its goal)

    PAUSED --> IDLE: Pause timer expired<br/>(1 replan tick ~ 3s)

    IDLE --> NAVIGATING: Next replan assigns<br/>new frontier
```

State Descriptions

State	Meaning	What Happens
IDLE	No active goal	Robot is available for assignment in next replan
NAVIGATING	Actively driving to a frontier	Nav2 is executing the path; coordinator monitors progress
PAUSED	Temporarily stopped for collision avoidance	Nav2 goal cancelled, zero velocity published, countdown timer running
WAITING_FOR_NAV	Goal sent, awaiting action server acceptance	Transitional state (brief)

Per-Robot Context Data

```
class RobotContext:
    name: str                # "robot1" or "robot2"
    state: RobotState        # Current state
    goal: np.array([x, y])   # Current target position (or None)
    goal_handle: GoalHandle  # Nav2 action goal handle (for cancellation)
    goal_start_time: Time    # When the current goal was assigned
    stuck_counter: int       # Consecutive replan ticks with no movement
    last_position: np.array  # Position at last replan (for movement detection)
    pause_timer: int         # Countdown ticks remaining while PAUSED
```

9. Stuck Detection & Recovery

Implemented in the replan loop.

Detection Logic

Every replan cycle (3 seconds), each navigating robot's position is compared to its position at the previous replan:

```
moved = np.linalg.norm(cur_pos - ctx.last_position)
if moved < 0.1:                # Less than 10cm of movement
    ctx.stuck_counter += 1
else:
    ctx.stuck_counter = 0      # Reset if robot moved

if ctx.stuck_counter >= self.stuck_threshold: # 3 consecutive ticks = 9 seconds
    # Robot is stuck!
    self._cancel_nav_goal(name)
    ctx.goal = None
    ctx.state = RobotState.IDLE
```

What Happens When Stuck

1. **Cancel the Nav2 goal** via `goal_handle.cancel_goal_async()`
2. **Send zero velocity** to immediately halt
3. **Clear the goal** and return to IDLE
4. **Reset the stuck counter**

On the next replan cycle, the robot will be assigned a new frontier — potentially in a different direction.

*[!NOTE] This stuck detection is **layered on top of** Nav2's own `SimpleProgressChecker` (which triggers recovery behaviors after 10s of no 0.5m movement). The frontier coordinator's stuck detection (9s, 0.1m threshold) is more aggressive because it also handles cases where Nav2's recovery succeeds but the robot still can't reach the frontier.*

Goal Retention Logic

Before declaring a robot "needing a new goal", the coordinator checks:

```
# Is the robot still far from its goal?
dist_to_goal = np.linalg.norm(cur_pos - ctx.goal)

# Is there still a frontier near the assigned goal?
still_frontier = any(
    np.linalg.norm(ctx.goal - f) < self.goal_retention_radius # 2.5m
    for f in frontiers)

# Minimum commitment time (prevent goal thrashing)
time_navigating = (now - ctx.goal_start_time).nanoseconds / 1e9

if dist_to_goal > 0.4 and (still_frontier or time_navigating < 10.0):
    continue # Keep navigating - don't reassign
```

This prevents premature goal switching. A robot keeps its goal if:

1. It hasn't reached it yet ($> 0.4\text{m}$ away), AND
2. Either the frontier still exists (within 2.5m) OR it hasn't been navigating for 10 seconds yet

10. Inter-Robot Collision Avoidance

Implemented in `_check_inter_robot_collision()` and the 10 Hz safety loop.

Detection

Every 100ms (10 Hz), both robot positions are looked up via TF:

```
dist = np.linalg.norm(pos_a - pos_b)
if dist < self.collision_radius: # 0.6m
    # Collision risk!
```

Resolution Strategy: Pause the One Further from Its Goal

```
dist_a = np.linalg.norm(pos_a - ctx_a.goal) if ctx_a.goal else inf
dist_b = np.linalg.norm(pos_b - ctx_b.goal) if ctx_b.goal else inf

if dist_a >= dist_b:
    to_pause = robot_a # Further from goal → pause
    to_keep = robot_b # Closer to goal → keep going
```

Rationale: The robot closer to its goal is more "committed" and will move out of the way faster. Pausing the further robot minimizes total navigation time.

Pause Actions

1. Cancel the paused robot's Nav2 goal
2. Send zero velocity (`Twist()`) to immediately stop
3. Set state to `PAUSED` with `pause_timer = 1` (one replan tick $\approx$ 3s)
4. Clear the goal (force fresh assignment on unpause)

Design History

*[!WARNING] An earlier version had more aggressive manual pausing, but this **fought with Nav2's recovery behaviors** (Spin/BackUp). When the coordinator paused a robot, Nav2 would detect "stuck" and trigger Spin — but the coordinator would immediately re-pause, creating a deadlock. The current design relies primarily on **spatial partitioning** (Section 7) to prevent crossing paths, with collision avoidance as a safety net.*

11. Nav2 Action Client Integration

Implemented in `_send_nav_goal()`, `_goal_response_callback()`, and `_goal_result_callback()`.

Goal Sending

```
goal_msg = NavigateToPose.Goal()
goal_msg.pose.header.frame_id = 'map'                # Global merged frame
goal_msg.pose.pose.position.x = float(target[0])
goal_msg.pose.pose.position.y = float(target[1])
goal_msg.pose.pose.orientation.w = 1.0               # Default heading (facing +X)

future = client.send_goal_async(goal_msg)
future.add_done_callback(self._goal_response_callback)
```

Goals are sent in the global `map` frame. The heading is always `w=1.0` (facing east) because frontier exploration doesn't care about final orientation — only reaching the frontier to observe new space.

Three-Phase Callback Chain

```
sequenceDiagram
    participant FC as Frontier Coordinator
    participant Nav2 as Nav2 BT Navigator

    FC->>Nav2: send_goal_async(NavigateToPose)
    Nav2-->>FC: Goal Response (accepted/rejected)

    alt Goal Accepted
        FC->>FC: Store goal_handle<br/>State → NAVIGATING
        Note over Nav2: Planning + Executing...
        Nav2-->>FC: Goal Result (succeeded/failed/canceled)

        alt Succeeded (status=4)
            FC->>FC: State → IDLE<br/>Clear goal
        else Failed (other status)
            FC->>FC: Blacklist frontier (30s)<br/>State → IDLE
        else Canceled (status=5)
            FC->>FC: State → IDLE<br/>(triggered by coordinator itself)
        end
    end

    else Goal Rejected
        FC->>FC: Blacklist frontier (30s)<br/>State → IDLE
    end
end
```

Stale Callback Protection

When a goal is preempted (cancelled and replaced), the old goal's result callback may fire after the new goal is already active. To prevent this:

```
def _goal_result_callback(self, future, robot_name, original_goal_handle):
    ctx = self.robots[robot_name]
    if ctx.goal_handle is not original_goal_handle:
        return # Ignore stale callback from preempted goal
```

Each callback captures a reference to the `goal_handle` at creation time. If the current handle doesn't match, the callback is from an old, cancelled goal and is safely ignored.

Goal Cancellation

```
def _cancel_nav_goal(self, robot_name):
    ctx = self.robots[robot_name]
    if ctx.goal_handle is not None:
        ctx.goal_handle.cancel_goal_async()
        ctx.goal_handle = None

    # Send zero velocity to immediately halt
    stop = Twist()
    self.cmd_pubs[robot_name].publish(stop)
```

Cancellation happens in two steps:

- 1. **Action cancellation:** Tells Nav2 to stop planning/executing
- 2. **Zero velocity:** Immediately halts the robot (Nav2's deceleration might be too slow)

12. RViz Visualization Pipeline

Frontier Markers

`_publish_frontier_markers()` publishes to `/frontiers_markers`:

Marker Type	Purpose	Color
<code>SPHERE</code>	Each detected frontier centroid	Green if assigned to robot1, Orange if robot2, Gray if unassigned
<code>TEXT_VIEW_FACING</code>	Label hovering above each sphere	Same color as sphere; shows <code>[Target: robotN]</code> or <code>[Unassigned]</code>

Goal Markers

`_publish_goal_markers()` publishes to `/frontier_goals_markers`:

Marker Type	Purpose	Color
<code>ARROW</code>	Downward-pointing arrow at each robot's current goal	Green for robot1, Orange for robot2
<code>TEXT_VIEW_FACING</code>	Label above arrow showing robot name and state	Same color; shows <code>robotN\n[NAVIGATING]</code>

Color Scheme

```
ROBOT_COLORS = {
    'robot1': ColorRGBA(r=0.0, g=1.0, b=0.2, a=0.9),    # Green
    'robot2': ColorRGBA(r=1.0, g=0.4, b=0.0, a=0.9),    # Orange
}
```

Marker Lifecycle

All markers have a lifetime of `replan_period + 2 = 5 seconds`. This ensures old markers disappear if the coordinator stops publishing (e.g., exploration complete). The `DELETEALL` action is sent at the start of each publish to clear stale markers.

13. All Tuning Parameters

Parameter	Value	Unit	Purpose	Source Line
<code>dedup_radius</code>	1.5	m	Merge centroids closer than this	L110
<code>goal_retention_radius</code>	2.5	m	Keep goal if any frontier is within this	L111
<code>min_commitment_time</code>	10.0	s	Minimum time before allowing goal switch	L112
<code>min_frontier_size</code>	15	px	Ignore frontier clusters smaller than this	L113
<code>goal_reached_threshold</code>	0.4	m	Consider goal reached when this close	L114
<code>min_goal_distance</code>	0.6	m	Skip frontiers closer than this	L115
<code>collision_radius</code>	0.6	m	Pause if robots are closer than this	L116
<code>pause_duration_ticks</code>	1	ticks	How many replan cycles to stay paused	L117
<code>separation_weight</code>	50.0	—	Inter-robot separation cost multiplier	L118
<code>region_weight</code>	50.0	—	Spatial partitioning cost multiplier	L119
<code>stuck_threshold</code>	3	ticks	Ticks of no movement before re-assigning	L120
<code>replan_period</code>	3.0	s	How often the full pipeline runs	L143
<code>blacklist_duration</code>	30.0	s	How long failed frontiers are blacklisted	L127

How Parameters Interact

```
flowchart LR
    subgraph "Frontier Quality"
        MFS["min_frontier_size<br/>(15 px)"]
        DR["dedup_radius<br/>(1.5 m)"]
        BD["blacklist_duration<br/>(30 s)"]
    end

    subgraph "Goal Stability"
        GRR["goal_retention_radius<br/>(2.5 m)"]
        MCT["min_commitment_time<br/>(10 s)"]
        ST["stuck_threshold<br/>(3 ticks = 9s)"]
    end

    subgraph "Multi-Robot Spread"
        SW["separation_weight<br/>(50)"]
        RW["region_weight<br/>(50)"]
        CR["collision_radius<br/>(0.6 m)"]
    end

    MFS -->|"Fewer, larger<br/>frontiers"| DR
    DR -->|"Fewer, spread-out<br/>candidates"| SW
    SW -->|"Robots choose<br/>distant goals"| RW
    RW -->|"Robots stay in<br/>their territory"| CR
    MCT -->|"Don't switch<br/>too often"| GRR
    ST -->|"Detect actual<br/>stuck, not slow"| BD
```

14. The Replan Loop — Putting It All Together

Here's the complete flow of one replan cycle, as implemented in `_replan()`:

```

flowchart TB
    START["Timer fires (every 3s)"] --> CHECK_MAP{"latest_map<br/>available?"}
    CHECK_MAP -->|No| SKIP["Skip cycle"]
    CHECK_MAP -->|Yes| CHECK_TF{"All robots<br/>have valid TF?"}
    CHECK_TF -->|No| WAIT["Wait (log warning)"]
    CHECK_TF -->|Yes| DETECT["Detect frontiers<br/>(morphological pipeline)"]

    DETECT --> DEDUP["Deduplicate<br/>(1.5m radius)"]
    DEDUP --> BLACKLIST["Filter out<br/>blacklisted frontiers"]
    BLACKLIST --> CHECK_EMPTY{"Any frontiers<br/>remaining?"}
    CHECK_EMPTY -->|No| DONE["Log: Exploration complete!"]
    CHECK_EMPTY -->|Yes| GET_POS["Get robot positions<br/>via TF"]

    GET_POS --> HANDLE_PAUSED["Handle paused robots<br/>(count down timers)"]
    HANDLE_PAUSED --> CHECK_GOALS["For each robot:<br/>Check goal validity"]

    CHECK_GOALS --> STUCK{"Stuck?<br/>(< 0.1m in 3 ticks)"}
    STUCK -->|Yes| CANCEL_STUCK["Cancel goal<br/>→ IDLE"]
    STUCK -->|No| REACHED{"Goal reached?<br/>(< 0.4m)"}
    REACHED -->|Yes| CANCEL_DONE["Clear goal<br/>→ IDLE"]
    REACHED -->|No| VANISHED{"Frontier gone?<br/>(no frontier within 2.5m<br/>AND > 10s navigating)"}
    VANISHED -->|Yes| CANCEL_VANISH["Cancel goal<br/>→ IDLE"]
    VANISHED -->|No| KEEP["Keep navigating<br/>(skip assignment)"]

    CANCEL_STUCK --> NEED_GOAL["Add to<br/>robots_needing_goal"]
    CANCEL_DONE --> NEED_GOAL
    CANCEL_VANISH --> NEED_GOAL

    NEED_GOAL --> REMOVE_TAKEN["Remove frontiers<br/>assigned to other robots"]
    REMOVE_TAKEN --> BUILD_COST["Build cost matrix<br/>dist + sep + region"]
    BUILD_COST --> ASSIGN["Greedy argmin<br/>assignment"]
    ASSIGN --> SEND_NAV["Send NavigateToPose<br/>action goal"]
    SEND_NAV --> PUBLISH_VIZ["Publish RViz markers"]

```

15. Comparison: Frontier Coordinator vs. `explore_lite`

The project includes an alternative exploration approach using `explore_lite` (from `m-explore-ros` 2). Here's how they compare:

Feature	Frontier Coordinator (custom)	explore_lite
Map Source	Merged <code>/map</code> (both robots' data)	Per-robot <code>global_costmap/costmap</code>
Cross-Robot Coordination	☐ Cost matrix with separation + region costs	☐ None — robots may explore same area
Spatial Partitioning	☐ Perpendicular bisector divides map	☐ None
Collision Avoidance	☐ 10 Hz safety loop with pause logic	☐ Relies on Nav2's obstacle layer
Nav2 Interface	Action client (<code>NavigateToPose</code>)	Topic (<code>/goal_pose</code>)
Goal Tracking	☐ Acceptance, rejection, result callbacks	☐ Fire-and-forget
Stuck Detection	☐ Custom 3-tick stuck counter	☐ Relies on Nav2's progress checker
Frontier Blacklisting	☐ 30s TTL blacklist	☐ None
Visualization	☐ Rich RViz markers (spheres, arrows, labels)	☐ Basic frontier visualization
Complexity	~824 lines of Python	Off-the-shelf (just launch params)

[!WARNING] Do NOT run both simultaneously! Both systems publish to `/robot/goal_pose` or send `NavigateToPose` actions, and will fight over navigation goals.

16. Evolution: v1 → v2 Improvements

The Frontier Coordinator has gone through a major revision. The current file header documents these improvements:

Feature	v1	v2
Assignment	Nearest frontier	Cost matrix with distance + separation + region costs
Spatial Partitioning	None	Perpendicular bisector with home positions
Collision Avoidance	None	10 Hz safety loop with pause/unpause
Nav2 Interface	Raw <code>/goal_pose</code> topic	<code>NavigateToPose</code> action client with callbacks
RViz Markers	Basic	Color-coded spheres, arrows, labels per robot
Stuck Detection	None	3-tick movement threshold
Frontier Blacklisting	None	30s TTL after Nav2 failure
Goal Retention	Reassign every cycle	Keep goal if frontier still exists + commitment time

17. Deprecated: Single-Robot Frontier Explorer

The project also contains a simpler, deprecated `frontier_exploration.py` — a single-robot frontier explorer kept for reference:

Key Differences from the Coordinator

Aspect	Single-Robot Explorer	Multi-Robot Coordinator
Robots	1 (hardcoded <code>robot1</code>)	2 (coordinated)
Frontier Detection	Canny edge detection → frontier mask	Morphological dilation → frontier mask (safer)
Selection	Largest frontier cluster	Cost-optimized assignment
Goal Interface	<code>/robot1/goal_pose</code> topic (fire-and-forget)	<code>NavigateToPose</code> action (tracked)
Obstacle Safety	None (raw edge detection)	Obstacle inflation (0.15m safety margin)
Lines of Code	~106	~824

The Canny-based approach in the deprecated version was replaced because it was less robust — Canny detects edges of free space but doesn't distinguish between free-obstacle edges (walls) and free-unknown edges (true frontiers) as cleanly as the morphological dilation approach.